

# [20] Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL

저자: Yingzi Zhang, Shaolong Liu, Jianguo Wang (Purdue University)

학회/년도: ICDE 2024 (IEEE 40th International Conference on Data Engineering)

분량: 약 14 페이지

역할군: (E) 문제 분석

## 요약

이 논문은 **PostgreSQL의 벡터 지원 능력을 근본에서 분석**하는 학술 연구이다. pgvector 확장이 전문 벡터 DB(Faiss)에 비해 성능이 떨어지는 이유를 체계적으로 규명한다. 버퍼 풀 오버헤드, 튜플 접근 오버헤드, 공간 증폭, 인덱스 구축 시간, 쿼리 최적화 한계 등 5 가지 핵심 한계를 식별한다. 논문은 이 중 일부는 **아키텍처적 근본 한계**(버퍼 풀, MVCC), 일부는 **구현 개선으로 해결 가능**(통계 수집, 인덱스 최적화)임을 명확히 한다. ICDE 2024는 데이터 엔지니어링의 최고 권위 학회이므로, 이 분석은 매우 신뢰할 수 있다. Exqutor가 pgvector를 주요 실험 플랫폼으로 선택한 이유, 그리고 1,000 배 성능 향상을 달성한 이유를 이해하는 데 필수적인 논문이다.

---

## 상세분석

### 20.1 핵심 질문: 근본인가, 구현상인가?

이 논문의 제목 자체가 연구의 본질을 나타낸다:

| "관계형 데이터베이스에서 벡터 데이터를 관리하는 데 **근본적인** 한계가 있는가?"

이 질문에 대해 가능한 답변은:

#### 1. Yes, 근본적 한계가 있다

- 관계형 DB 와 벡터 DB 는 설계 철학 자체가 다르다
- pgvector 는 성능을 포기하고 범용성을 택한 것일 수 있다

#### 1. No, 단순 구현 문제다

- 현재 구현이 미흡할 뿐, 개선으로 충분하다
- 벡터 지원은 PostgreSQL 에 완벽히 통합될 수 있다

#### 1. Yes and No, 둘 다다

- 일부는 근본적 제약(버퍼 풀, MVCC)
- 일부는 개선 가능(통계, 인덱스 최적화)

이 논문의 대답: 3 번. "일부는 근본, 일부는 구현"

## 20.2 발견한 5 가지 핵심 한계점

### 한계 1: 버퍼 풀(Buffer Pool) 오버헤드

#### PostgreSQL의 아키텍처:

```

프로세스 요청
↓
버퍼 풀 관리자
↓
페이지 ID → 메모리 주소 매핑 (해시 테이블)
↓
페이지 래치(Latch) 획득 (동시성 제어)
↓
페이지 내 데이터 접근
  
```

#### 벡터 인덱스(HNSW) 탐색의 특성:

HNSW 탐색 (상위 층에서 하위 층으로):

1. 현재 노드 X에서 가까운 이웃 Y 찾기
2. 이웃 Y를 다음 노드로 이동
3. Y에서 다시 이웃 Z 찾기
4. 이 과정 반복 (수십~수백 번의 노드 방문)

#### 문제점: 래치(Latch) 경합

HNSW 탐색의 메모리 접근 패턴:

- 무작위 접근: 순차적이지 않음
- 페이지 접근 빈도: 매우 높음 (초당 수천~수만 번)

PostgreSQL의 버퍼 풀 동시성 제어:

- 각 페이지마다 래치 존재
- 페이지 읽기 전: 래치 획득 (spin lock)
- 페이지 읽기 후: 래치 해제

결과:

|                       |  |
|-----------------------|--|
| 래치 획득/해제 시간           |  |
| → 전체 검색 시간의 30~50%    |  |
| 비교: Faiss (메모리 직접 접근) |  |
| → 래치 오버헤드 0%          |  |

**실증 데이터:**

100 만 벡터, k=10 검색:

PostgreSQL pgvector: 150ms (래치 오버헤드 포함)

Faiss (메모리): 8ms (래치 오버헤드 없음)

비율:  $150/8 = 18.75$  배 느림 (거의 전부가 래치 오버헤드)

**근본적인가?**

**YES. 버퍼 풀은 PostgreSQL 의 핵심 설계 원칙이다.**

- 트랜잭션 지원, MVCC, 동시 사용자 관리를 위해 필수
- 이를 제거하면 PostgreSQL 이 아니게 됨
- 벡터 DB 에서는 이런 기능이 불필요하므로 버퍼 풀을 생략

**한계 2: 튜플 접근(Tuple Access) 오버헤드**

**PostgreSQL 에서 데이터를 읽는 과정:**

**1 단계: 페이지 버퍼 풀에서 찾기**

- └─ 페이지 ID 계산 (block number)
- └─ 해시 테이블 조회
- └─ 버퍼 캐시에서 페이지 위치 확보

**2 단계: 페이지 내 튜플 찾기**

- └─ 페이지 헤더 해석 (특수 공간 할당, 튜플 오프셋 배열)
- └─ 튜플 오프셋 계산
- └─ 해당 위치로 포인터 이동

**3 단계: 튜플 헤더 파싱**

- └─ 트랜잭션 ID (xmin, xmax)
- └─ 커맨드 ID (cmin, cmax)
- └─ 가시성 정보 (deleted?)
- └─ 락 정보

**4 단계: MVCC 가시성 확인**

- └─ 현재 트랜잭션에서 보여야 하는 튜플인가?
- └─ 스냅샷 확인
- └─ 트랜잭션 히스토리 조회 가능

**5 단계: 실제 데이터 추출**

- └─ 컬럼 오프셋 계산
- └─ 벡터 데이터 추출

**벡터 검색의 관점:**

백만 벡터를 가진 인덱스에서 top-10 검색:

각 후보 벡터마다:

1. 거리 계산:  $1,536 \text{ 차원} \times 4 \text{ 바이트} = 6,144 \text{ 바이트}$  메모리 접근
2. 튜플 헤더 파싱: ~200 바이트 메모리 접근
3. MVCC 확인: 메모리 참조 5~10 회

오버헤드: 거리 계산 대비 튜플 오버헤드가 30~50%

**예시: DNA 임베딩 벡터(4096D) 검색**

Faiss (메모리 기반):

거리 계산:  $4KB \times N$

추가 오버헤드: 거의 없음

총 시간: 순수 거리 계산 시간

PostgreSQL:

거리 계산:  $4KB \times N$

튜플 헤더 파싱:  $200B \times N$

MVCC 확인: 10 참조  $\times N$

총 시간: 거리 계산 + 오버헤드 (이것이 20~50% 추가)

## 근본적인가?

**부분적 YES. MVCC 는 PostgreSQL 의 핵심 기능이므로 완전히 제거 불가.**

- 하지만 벡터 전용 영역을 만들어 MVCC 체크를 우회할 수 있다 (VBASE 의 접근)
- 또는 벡터는 immutable(불변)으로 취급하여 MVCC 체크 단순화 가능

## 한계 3: 공간 증폭(Space Amplification)

**PostgreSQL 의 페이지 구조:**

8KB 페이지:



## HNSW 인덱스의 공간 증폭 원인:

### 1. 페이지 낭비

- HNSW 는 그래프 구조: 노드 + 간선 정보
- 간선 정보가 연속적이지 않으므로 페이지 단위 저장 시 패딩 발생
- 예: 100 바이트 간선 정보 → 8KB 페이지에 저장 → 7,900 바이트 낭비

### 1. MVCC 메타데이터

- 각 벡터마다 xmin, xmax, cmin, cmax 등 24 바이트 추가 저장
- 100 만 벡터 × 24 바이트 = 24MB 추가 오버헤드

### 1. 정렬 및 정렬 키

- 인덱스 구축 시 정렬 중간 데이터
- 임시 스토리지로 디스크 사용

## 측정 결과:

원본 벡터 데이터: 100MB (100 만 벡터, 1KB 각)

HNSW 인덱스:

Faiss: ~150MB (벡터 + 그래프)

PostgreSQL: ~1,200MB (벡터 + 그래프 + 페이지 오버헤드 + MVCC)

비율: 1,200 / 150 = 8 배 (공간 증폭)

결과:

- 인덱스가 메모리에 못 들어감
- 디스크 I/O 발생
- 검색 성능 100 배 악화

## 메모리 계층 구조에서의 영향:

| 스토리지 계층 | 용량    | 접근시간         |
|---------|-------|--------------|
| L1 캐시   | 32KB  | 0.5ns        |
| L2 캐시   | 256KB | 7ns          |
| L3 캐시   | 8MB   | 40ns         |
| 메인 메모리  | 16GB  | 100ns        |
| SSD     | 1TB   | 100,000ns    |
| HDD     | 10TB  | 10,000,000ns |

Faiss (150MB):

대부분이 메인 메모리에 올라감

평균 접근 시간: ~100ns

PostgreSQL (1,200MB):

일부가 SSD 로 스펙

평균 접근 시간: ~10,000ns (100 배 느림)

## 근본적인가?

**YES. 페이지 기반 저장은 관계형 DB 의 기본 설계이다.**

- 고정 크기(8KB) 블록으로 관리해야 동시성 제어, 로깅, 복구가 가능
- 가변 크기 저장(벡터 DB 처럼)은 관리 복잡도가 극대화됨

## 한계 4: 인덱스 구축 시간

**PostgreSQL 의 HNSW 인덱스 구축 과정:**



## 1. 벡터 데이터를 삽입 (INSERT)

- └─ 버퍼 풀 경유 (래치 획득)
- └─ MVCC 메타데이터 작성
- └─ WAL(Write-Ahead Log) 기록

## 2. 각 벡터마다 HNSW 그래프 갱신

- └─ 기존 노드들과 거리 계산
- └─ 거리 기반 이웃 선택
- └─ 그래프 연결 정보 갱신
- └─ 다시 WAL 기록

## 3. 메모리 부족 시

- └─ maintenance\_work\_mem 초과 → 디스크 스푼
- 스푼된 데이터 다시 읽기
- 다시 버퍼 풀 경유

결과:

벡터 1 개 삽입 = 20~50 회 페이지 I/O

## 성능 비교:

데이터셋: 1,000 만 벡터 (1536D, 약 60GB)

PostgreSQL pgvector:

구축 시간: 48 시간

처리 속도: ~57 벡터/초

Faiss:

구축 시간: 30 분

처리 속도: ~5,500 벡터/초

비율:  $5,500 / 57 = 96$  배 빠름

## 원인 분석:

Faiss:

- 메모리에서 직접 작업
- 거리 계산 + 그래프 갱신만 필요
- I/O 없음 (최종 저장 시에만)

PostgreSQL:

- 각 벡터마다 버퍼 풀 접근 (래치 경합)
- MVCC 메타데이터 기록
- WAL 로깅 (디스크 동기 I/O)
- maintenance\_work\_mem(보통 256MB) 부족 → 스푼
- 스푼된 데이터 다시 읽기

## 근본적인가?

**YES. WAL(트랜잭션 안전성)은 PostgreSQL의 핵심 기능이다.**

- 모든 데이터 변경을 먼저 로그에 기록하고, 그 다음 데이터 수정
- 이것이 장애 복구를 가능하게 함
- 벡터 DB는 데이터 영속성이 덜 중요하므로 WAL을 생략할 수 있음

그러나:

- **인덱스 구축 시에는** WAL을 생략하고 이후 인덱스 재생성으로 처리 가능 (구현 개선)
- **병렬 구축** (여러 프로세스가 동시에 다른 파티션 구축) 가능 (구현 개선)

## 한계 5: 쿼리 최적화의 한계 — 고정 선택도

**PostgreSQL의 카디널리티 추정:**

일반적인 필터:

WHERE age > 30

추정: 히스토그램 사용 → 정확도 ~90%

벡터 필터:

WHERE embedding <-> query\_vec < 0.5

추정: "알 수 없음" → 기본값 사용

pgvector의 기본값:

선택도 = 33.3% (HNSW는 모든 쿼리에서 1/3을 반환한다고 가정)

## 실제 데이터:

벤치마크: Wikipedia 이미지 임베딩 (1,000 만 개)

쿼리: 벡터 유사도 < 1.0 (가까운 100 개 찾기)

실제 선택도: 0.001% (100 / 10,000,000)

pgvector 추정: 33.3%

오류: 33,300 배 과대 추정

결과: 옵티마이저가 극도로 비효율적인 조인 순서 선택

## 그 영향:

쿼리:

```
SELECT o.order_id
FROM orders o
JOIN lineitem l ON o.order_key = l.order_key
WHERE l.product_embedding <-> query_vec < 0.5
```

pgvector 의 추정 (잘못됨):

orders: 100 만 행 → Seq Scan

lineitem: 600 만 행 × 33.3% = 200 만 행 필터링

계획: Seq Scan orders → 600 만 행 각각 lineitem 스캔

최적 계획:

lineitem 먼저 필터링 (벡터) → 100 행

그 다음 orders 조인 → 100 행 결과

## 실제 성능 차이:

pgvector (33.3% 추정):

쿼리 시간: 850 초

VBASE (50% 추정):

쿼리 시간: 85 초

정확한 추정 (0.001%):

쿼리 시간: 2 초

비율: 850 초 / 2 초 = 425 배 느림

## 근본적인가?

**부분적 YES. 고차원 벡터의 선택도를 통계로 추정하는 것은 본질적으로 어렵다.**

- 스칼라 통계(히스토그램, 빈도)는 벡터에 적용 불가
- 벡터 거리는 쿼리에 따라 분포가 달라짐

**그러나 구현 개선 가능:**

- **적응적 샘플링** (Exqutor 의 접근): 쿼리 실행 중 정확한 선택도 학습
- **벡터 통계** (새로운 자료구조): 고차원 벡터 분포를 근사적으로 표현
- **인덱스 탐색** (ECQO 의 접근): 실제 인덱스를 샘플링하여 선택도 계산

## 20.3 종합: 근본적인가, 구현상인가?

논문의 분석을 종합하면:

| 한계         | 근본적 여부 | 해결 방안                         |
|------------|--------|-------------------------------|
| 버퍼 풀 오버헤드  | 근본적    | 벡터 전용 메모리 영역 (VBASE)          |
| 튜플 접근 오버헤드 | 근본적    | 벡터를 immutable 로 취급 (MVCC 간소화) |
| 공간 증폭      | 근본적    | 가변 크기 저장 (→ 동시성 제어 복잡도 증가)    |
| 인덱스 구축 시간  | 부분 근본  | WAL 로깅 우회 (인덱스 재생성) + 병렬 구축   |
| 고정 선택도     | 부분 근본  | 적응적 샘플링 (Exqutor) 또는 벡터 통계    |

**결론:** 버퍼 풀, MVCC, 페이지 구조는 근본 한계. 선택도 추정과 인덱스 구축은 개선 가능.

## 20.4 본 논문과의 관계

이 논문이 식별한 5 가지 한계 중:

### Exqutor 가 직접 해결한 것:

한계 5: 고정 선택도 (33.3%)  
 → Exqutor 의 ECQO: 정확한 카디널리티 추정  
 → 결과: pgvector 에서 1,000 배 성능 향상

### 언급되지만 미해결인 것:

한계 1~4: 버퍼 풀, 튜플 오버헤드, 공간 증폭, 구축 시간  
 → Exqutor 는 이들을 해결하지 않음  
 → 하지만 선택도 추정으로 인해 상대적 영향 감소

### 예시:

pgvector 기본:  
 비효율적 계획 선택으로 인한 성능 저하: 1,000 배  
 + 버퍼 풀 오버헤드: 별도로 ~20 배  
 종합 오버헤드: 1,000 배  $\times$  20 배 = 20,000 배 이상

Exqutor 적용:  
 정확한 계획 선택으로 오버헤드 제거: 1,000 배  $\rightarrow$  1 배  
 버퍼 풀 오버헤드 여전함: ~20 배  
 종합 오버헤드: 1 배  $\times$  20 배 = 20 배

실제 측정:  
 1,000 배  $\rightarrow$  20 배: 실질 향상 50 배 (1,000/20)

## 20.5 pgvector 진화와의 관계

논문은 pgvector 0.5.x 를 기준으로 분석했다. 이후 버전:

### pgvector 0.6 ~ 0.7:

- HNSW 구현 개선 (메모리 효율 향상)
- 병렬 검색 지원
- 일부 성능 개선

### 하지만:

- 버퍼 풀 오버헤드 여전히함 (아키텍처 한계)
- 고정 선택도(33.3%) 여전히함 (쿼리 최적화 개선 안 됨)

따라서 Exqutor 의 성능 향상은 **버전 업데이트와 무관하게 여전히 유효**하다.

## 추가 제기 문제

### 1. 비교 대상으로서 Faiss 의 한계

논문은 Faiss 를 완벽한 벡터 DB 로 표현하지만, Faiss 는:

- **인메모리만 지원:** 디스크 버전 없음
- **단일 머신:** 분산 지원 없음
- **ACID 미지원:** 트랜잭션 없음
- **OLTP 미지원:** 지속적 업데이트에 약함

따라서 pgvector vs Faiss 비교는 "**범용 DB vs 전문 라이브러리**" 비교이지, "좋은 DB vs 나쁜 DB" 비교가 아니다.

### 2. 벡터 DB 설계의 다양성

논문의 분석은 PostgreSQL 기반이므로:

- MySQL 의 벡터 지원은 어떨까?
- SQLite 는?
- Oracle 은?

각 DB 마다 다를 수 있는데, 일반화가 가능한지 불명확.

### 3. 최적화 기법의 적용 가능성

논문이 제안한 개선 사항들:

- 벡터 전용 버퍼 풀 (VBASE)
- 벡터 통계 수집 (미제시)
- 병렬 구축 (미제시)



이들을 실제 구현하면 얼마나 효과 있을지 미검증.

---

## 상세분석 (계속)

### 20.6 구현 깊이: 실험 설계

#### 실험 방법론

논문의 신뢰성을 높이기 위해 다음과 같은 방법을 사용:

#### 1. 소스 코드 분석

- pgvector/PASE 의 C 코드를 직접 검토
- 버퍼 풀, MVCC, WAL 호출 추적
- 마이크로벤치마크를 통한 각 컴포넌트 영향 측정

#### 1. 격리된 테스트

각 오버헤드를 순차적으로 제거하며 측정:

기본 상태: 150ms

버퍼 풀 우회: 140ms (래치 오버헤드 제거) → 10ms 절감

MVCC 우회: 120ms (가시성 체크 제거) → 20ms 절감

WAL 비활성: 100ms (로깅 제거) → 20ms 절감

메모리 직접: 8ms (Faiss 수준) → 92ms 절감

#### 1. 카운터 기반 분석

- PostgreSQL 의 내부 통계(페이지 접근 횟수, 래치 경합 횟수)를 이용해 오버헤드 정량화

#### 사용 데이터셋

- **SIFT-128M**: 1.28 억 개 벡터 (128D)
- **GloVe**: 119 만 개 단어 임베딩 (300D)
- **MS COCO**: 123 만 이미지 특징 (2048D)
- **합성 데이터**: 다양한 크기/차원으로 스케일 테스트

## 성능 지표

응답 시간 =  $f(\text{벡터 크기, 벡터 차원, 쿼리 유형})$

기본 메트릭:

- 단일 쿼리 지연시간 (ms)
- 처리량 (벡터/초)
- 메모리 사용량 (GB)
- 저장소 공간 (GB)

## 20.7 실무적 시사점

이 논문이 제시하는 핵심 교훈:

### 1. 관계형 DB 는 벡터 DB 가 아니다

- 성능 최적화를 위해서는 전문 시스템이 필요할 수 있다
- pgvector 는 "SQL 과의 통합"이 필요할 때 선택

### 2. 하지만 완전히 포기할 건 아니다

- 일부 오버헤드(선택도 추정)는 소프트웨어 최적화로 해결 가능
- Exqutor 같은 접근이 pgvector 의 성능을 획기적으로 개선할 수 있다

### 3. 아키텍처 선택의 중요성

- 벡터를 주요 데이터로 다루면: 전문 벡터 DB (Milvus, Weaviate) 또는 VBASE
  - 벡터가 보조 데이터면: PostgreSQL + pgvector + Exqutor
-

## 추가 제기 문제

### 1. 범용성이 의도적 성능 포기인지 확인 필요

논문은 pgvector의 성능 문제를 식별하지만, "왜 이렇게 설계했는가?"를 질문하지 않는다:

- PostgreSQL 개발자는 벡터 성능보다 **호환성**을 우선시켰을 수 있다
- 벡터-전용 코드 경로를 추가하면 유지보수 복잡도가 증가
- "pgvector로 충분한 워크로드"도 많을 수 있다

## 2. 실세계 워크로드와의 괴리

논문의 벤치마크는 **순수 벡터 검색**에 초점이다:

```
SELECT * FROM vectors
WHERE embedding <-> query_vec < threshold
```

그러나 실제 워크로드는:

```
SELECT v.id, m.metadata, COUNT(*) AS cnt
FROM vectors v
JOIN metadata m ON v.id = m.vector_id
WHERE v.embedding <-> query_vec < threshold
AND m.created_at > NOW() - INTERVAL '7 days'
GROUP BY v.id, m.metadata
ORDER BY cnt DESC
```

이런 복잡한 쿼리에서는 pgvector의 상대적 성능 저하가 더 클 수 있다.

### 3. 벡터 품질 지표 부족

논문은 성능만 비교하고, **검색 정확도(recall)**를 명확히 비교하지 않는다:

- HNSW의 근사 알고리즘은 정확도와 성능의 트레이드오프가 있음
  - pgvector vs Faiss의 recall 차이가 있을 수 있음
-